

ENGINEERING

Coding Standards

Language style, naming, branching, code review, testing and security rules for all Nexora repositories

Document ID	NX-ENG-003	Version	2.6
Document owner	Engineering Excellence Guild	Effective date	1 February 2026
Applies to	All engineers and contractors contributing to Nexora repositories	Classification	Internal - Confidential

Abstract. This document sets the mandatory coding standards for Nexora Technologies. It defines language versions and tooling for Python and TypeScript, naming conventions, the Git branching model, commit message format, pull request and code review rules, minimum test coverage by module tier, logging practices and security requirements. The aim is consistent, readable and safe code that any engineer can review and maintain.

Keywords: coding standards, Python, TypeScript, PEP 8, Git branching, code review, unit testing, secrets management

1. Languages and Tooling

Table 1. Approved languages and mandatory tools.

Area	Standard	Mandatory tooling
Python (backend, data, AI)	Python 3.11 or newer; PEP 8 with a 100-character line limit; type hints on all public functions	black, ruff, mypy (strict on new modules), pytest
TypeScript (frontend, Node)	TypeScript 5.x with strict mode; no implicit any	ESLint, Prettier, Jest or Vitest
SQL	Uppercase SQL keywords; schema migrations via Alembic or Flyway	sqlfluff
Infrastructure	Terraform 1.6+; all resources tagged with owner, service and environment	tflint, terraform validate
API design	REST conventions in the API Documentation (NX-ENG-001); OpenAPI 3.1 specification kept in the repository	spectral

Formatters and linters run as pre-commit hooks and again in CI; formatting comments in reviews are therefore discouraged. Python dependencies are pinned in `requirements.txt` or `pyproject.toml` with a lock file.

2. Naming Conventions

Table 2. Naming rules.

Element	Convention	Example
Python modules and functions	snake_case	load_customer_profile()
Python classes	PascalCase	TicketRouter
Constants	UPPER_SNAKE_CASE	MAX_RETRY_COUNT
TypeScript variables and functions	camelCase	fetchTicketList
TypeScript components and types	PascalCase	TicketTable
Git branches	type/JIRA-ID-short-description	feature/NX-1234-add-csat-filter
Database tables and columns	snake_case, plural table names	support_tickets.created_at

```
# Not acceptable
def proc(d):
    for r in rules:
        if r(d): return r.q
```

```
# Preferred
def route_ticket(ticket: Ticket, rules: list[Rule]) -> Queue:
    """Return the queue whose rule first matches the ticket."""
    for rule in rules:
        if rule.matches(ticket):
            return rule.queue
    raise NoMatchingRuleError(ticket.id)
```

3. Git Branching Model

Nexora uses a simplified GitFlow model (Figure 1). The main branch always mirrors production and is protected; direct pushes are not allowed. Engineers create feature/* branches from develop, open a pull request, and merge after review. Release branches stabilise a version, and urgent production fixes use hotfix/* branches that are merged to both main and develop.

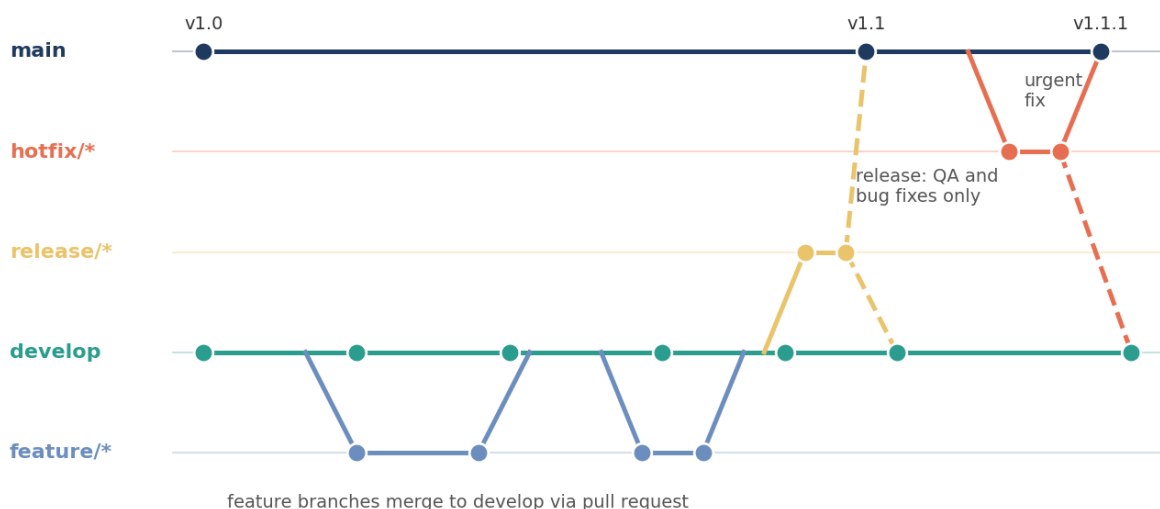


Figure 1. Branching model: main, develop, feature, release and hotfix branches.

3.1 Commit Messages

Commits follow the Conventional Commits format: `type(scope): summary`, using the types `feat`, `fix`, `docs`, `refactor`, `test` and `chore`. The summary is at most 72 characters and written in the imperative mood, for example `fix(tickets): prevent duplicate creation on retry`.

4. Code Review Rules

Table 3. Pull request and review requirements.

Rule	Requirement
Reviewers	At least 1 approval; 2 approvals for changes to authentication, payments, permissions or data-deletion code
Size	Keep pull requests under 400 changed lines where possible; split larger changes
Review turnaround	First response within 1 business day
Author checklist	Tests added, documentation updated, no debug code, linked Jira ticket, screenshots for UI changes
Merge conditions	All CI checks green, no unresolved comments, branch up to date with target
Merge method	Squash merge into develop; merge commit for release and hotfix branches

5. Testing

Every feature and bug fix must include automated tests. Minimum line coverage depends on the risk tier of the module (Figure 2); CI fails when coverage of changed files drops below the tier threshold. Tests must be deterministic, run without network access to external services, and complete within 10 minutes for the full unit suite.

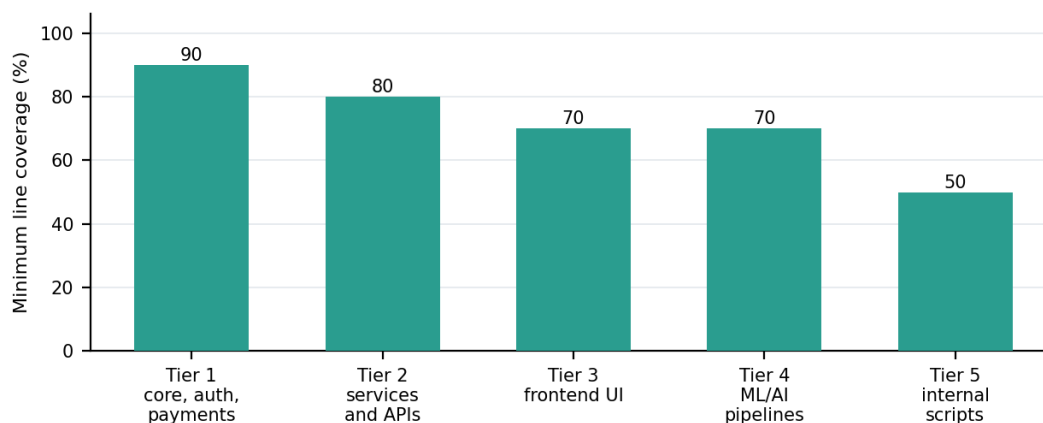


Figure 2. Minimum unit-test line coverage by module tier.

For AI and machine-learning code, unit tests cover data validation and pre-processing, and evaluation scripts run in CI on a fixed sample dataset. Prompt templates and retrieval configurations are stored in version control and change through pull requests like any other code.

6. Logging, Errors and Observability

- Use structured JSON logs with the fields `timestamp`, `level`, `service`, `request_id` and `message`.
- Never log passwords, tokens, API keys or full customer content; mask e-mail addresses and phone numbers.
- Catch specific exceptions; never use a bare `except`. Return errors in the API error model.
- Every service exposes `/health` and `/ready` endpoints and publishes RED metrics (rate, errors, duration).

7. Security Requirements

- **No secrets in code.** Credentials are stored in AWS Secrets Manager. Secrets found in a repository must be rotated immediately and reported to Security within 1 hour.
- Dependency scanning (Dependabot), static analysis (CodeQL) and container scanning (Trivy) run on every pull request; critical and high findings block the merge.
- Validate and sanitise all external input; use parameterised queries only.
- Use company-approved AI coding assistants only, and never include customer data or credentials in prompts (see Employee Handbook, Code of Conduct).

8. Documentation

Each repository contains a `README.md` with purpose, setup, run and test instructions, an owner and an on-call contact. Public functions use Google-style docstrings. Architecture decisions are recorded as short ADR documents in `/docs/adr`.

9. References

[1] PEP 8: Style Guide for Python Code.

- [2] Conventional Commits 1.0.0 specification.
- [3] Nexora Deployment SOP, NX-ENG-002.
- [4] Nexora API Documentation, NX-ENG-001.
- [5] OWASP Application Security Verification Standard (ASVS) 4.0.

10. Revision History

Table 4. Document revision history.

Version	Date	Author	Summary of change
2.0	01 Jun 2025	Engineering Excellence Guild	Adopted tiered coverage and GitFlow model
2.5	01 Nov 2025	Engineering Excellence Guild	Added guidance for AI/ML code and prompt templates
2.6	01 Feb 2026	Engineering Excellence Guild	Raised Tier 1 coverage to 90%; added secrets-leak reporting SLA